

SIMATS ENGINEERING

Practical Record — Virtualization Lab Exercise

Course: Computer Networks

Experiment: Demonstrate Virtualization by Installing a Type-2 Hypervisor on the Host Device, and Create & Configure a Virtual Machine (VM) with a Guest Operating System (Windows/Linux) using Oracle VirtualBox.

Aim

To understand the concept of virtualization by installing a Type-2 (hosted) hypervisor — Oracle VM VirtualBox — on the host operating system, and to create, configure, and boot a virtual machine running a guest operating system, thereby demonstrating how a single physical machine can host an independent, isolated computing environment on top of its existing OS.

Theory

Virtualization

Virtualization is the process of creating a software-based (virtual) version of a physical resource — such as a server, storage device, or an entire computer — so that multiple isolated operating environments can run on one physical machine. It improves hardware utilization, isolates workloads, and allows different operating systems to run side-by-side without interfering with each other.

Hypervisors — Type 1 vs Type 2

Aspect	Type-1 (Bare-Metal) Hypervisor	Type-2 (Hosted) Hypervisor
Where it runs	Directly on physical hardware, with no underlying OS	As an application on top of an existing host operating system
Performance	Higher — direct hardware access	Slightly lower — depends on host OS scheduling
Typical use	Data centres, enterprise servers	Personal computers, labs, testing/development
Examples	VMware ESXi, Microsoft Hyper-V (server), Citrix XenServer	Oracle VM VirtualBox, VMware Workstation/Player

This experiment uses **Oracle VM VirtualBox**, a free and open-source **Type-2 hypervisor** that installs like a

normal application on the host operating system (Windows/Linux/macOS) and lets that host create and run one or more guest virtual machines.

Software / Hardware Required

- Host machine with virtualization support enabled in BIOS/UEFI (Intel VT-x or AMD-V).
- Host Operating System — Windows 10/11 or a Linux distribution (e.g., Ubuntu).
- Oracle VM VirtualBox installer (downloaded from the official VirtualBox website).
- VirtualBox Extension Pack (optional, for USB 2.0/3.0 and other advanced features).
- A guest OS installation ISO image (e.g., Ubuntu Desktop ISO or Windows ISO).
- Minimum 4 GB RAM on host (8 GB or more recommended), and at least 25 GB free disk space for the VM.

Procedure

Part A — Installing the Type-2 Hypervisor (Oracle VirtualBox)

- 1 Open a web browser on the host machine and go to the official VirtualBox download page ([virtualbox.org](https://www.virtualbox.org) → Downloads).
- 2 Select the installer that matches the host operating system — e.g., 'Windows hosts' (.exe) if the host runs Windows, or the appropriate .deb/.rpm package (or use the distribution's package manager) if the host runs Linux.
- 3 Run the downloaded installer. On Windows, launch the .exe and proceed through the setup wizard, accepting the default components (USB support, networking, Python support) unless a custom setup is required. On Linux, install via terminal, for example: `sudo apt update && sudo apt install virtualbox`.
- 4 Accept any prompts about a brief network interruption (VirtualBox installs virtual network adapters) and allow the installation to complete.
- 5 Launch **Oracle VM VirtualBox Manager** once installation finishes, and confirm the application opens successfully — this confirms the Type-2 hypervisor is installed and running as a normal application on top of the host OS.
- 6 (Optional) Download and install the matching **VirtualBox Extension Pack** from the same website through File → Tools → Extension Pack Manager, to enable USB 3.0, PXE boot for Intel cards, and disk encryption support.

Part B — Creating and Configuring a New Virtual Machine

- 1 In the VirtualBox Manager window, click **New** to start the Create Virtual Machine wizard.
- 2 Enter a **Name** for the VM (e.g., 'Ubuntu-Guest' or 'Windows-Guest'); VirtualBox auto-detects the **Type** (Linux/Windows) and **Version** from the name — verify or correct these fields, and choose the **Machine Folder** where VM files will be stored.
- 3 Assign **Memory Size (RAM)** to the VM — e.g., 2048 MB for a lightweight Linux guest or 4096 MB for Windows — keeping enough RAM free for the host OS itself.
- 4 Create a new **Virtual Hard Disk**: choose 'Create a virtual hard disk now', select the file type **VDI (VirtualBox Disk Image)**, choose **Dynamically allocated** storage (so the file grows only as needed), and set a disk size (e.g., 25 GB).
- 5 After the VM is created, select it in the Manager list and click **Settings** to fine-tune configuration: under **System**, set processor core count and enable I/O APIC if needed; under **Display**, increase Video Memory and enable 3D acceleration if required; under **Storage**, attach the guest OS **ISO image** to the virtual optical drive; under **Network**, set Adapter 1 to **NAT** (default, for straightforward internet access) or **Bridged Adapter** (to give the VM its own IP on the LAN).
- 6 Click **Start** to power on the VM. With the ISO attached, the VM boots from it and launches the guest operating system installer.
- 7 Follow the guest OS installation wizard exactly as on a physical machine — select language, partition the virtual disk, create a user account, and complete installation.
- 8 Once installation finishes, remove/eject the ISO from the virtual drive (Devices → Optical Drives → Remove disk) and restart the VM so it boots from the virtual hard disk into the newly installed guest OS.
- 9 (Optional but recommended) Install **Guest Additions** (Devices → Insert Guest Additions CD image) inside the guest OS to enable shared clipboard, shared folders, better display resolution, and improved mouse/keyboard integration between host and guest.

Part C — Verifying the Virtualized Environment

- 1 Confirm the guest OS desktop/login screen loads inside the VM window, independent of the host OS.
- 2 Check that host and guest can each be operated independently (e.g., open applications in the guest without affecting the host, and vice-versa), demonstrating isolation.
- 3 Optionally verify networking by pinging an external address (e.g., ping 8.8.8.8) from inside the guest OS to confirm the virtual NAT/Bridged network adapter is functioning.
- 4 Use **VirtualBox Manager** → **Snapshot** to take a snapshot of the running VM, demonstrating another key virtualization feature — the ability to save and restore an exact VM state.

Result

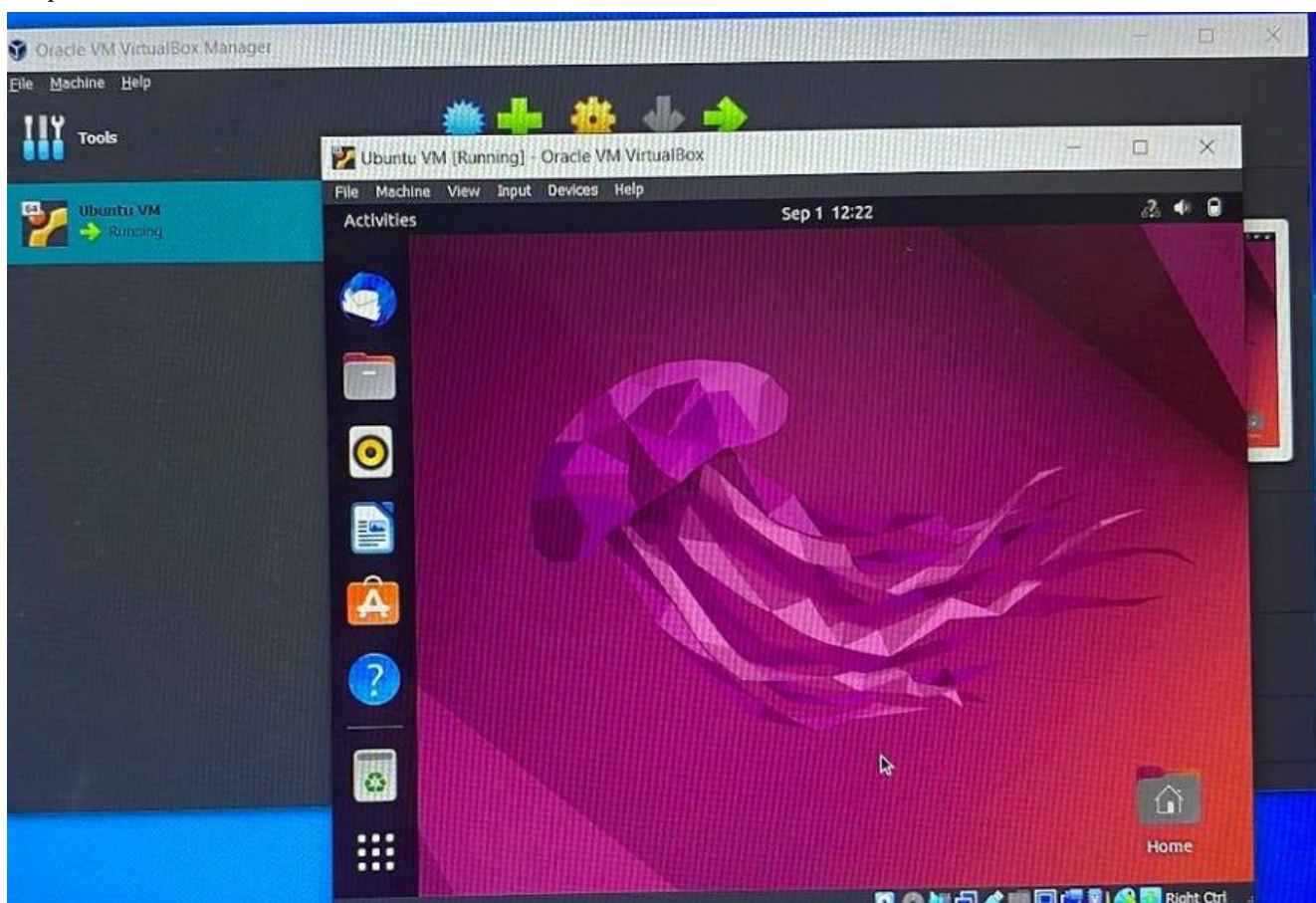
A Type-2 hypervisor (Oracle VM VirtualBox) was successfully installed on the host operating system. A new virtual machine was created, configured with the required virtual CPU, RAM, virtual hard disk, and network adapter, and a guest operating

system was installed and booted successfully inside it. The guest OS ran as an independent, isolated environment on top of the host OS, confirming that virtualization was successfully demonstrated.

Observation / Inference

- The host operating system continued to run normally while the guest OS operated inside its own virtual machine window, proving process- and resource-level isolation between host and guest.
- VirtualBox (a Type-2 hypervisor) relies on the host OS for hardware access (via drivers), unlike a Type-1 hypervisor, which would run directly on hardware without a host OS underneath it.
- Virtual hardware resources (RAM, virtual disk, virtual network adapter) allocated in VM Settings directly determine the guest OS's performance and connectivity, illustrating how a hypervisor abstracts and partitions physical resources among virtual machines.
- Features such as snapshots and Guest Additions demonstrate practical benefits of virtualization: safe experimentation (revert to a saved state) and improved host-guest integration.

Output:



Conclusion: Virtualization using a Type-2 hypervisor was successfully demonstrated by installing VirtualBox on the host OS and creating a fully functional, isolated guest virtual machine.